



WELCOME



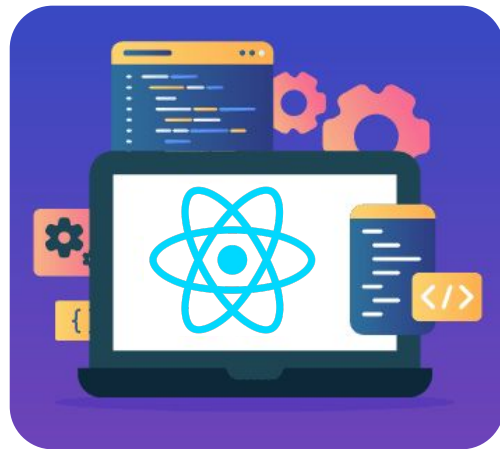


Components and Props

Recap

Introduction to React JS

- ✓ MERN Stack
 - Introduction to React JS
- ✓ React
 - `React.createElement()`
 - `ReactDOMClient.createRoot()`
 - `root.render()`
- ✓ JSX
 - Embedding Expressions
 - Nesting elements



Agenda for Today's Session

Components and Props

Starting the Application
React Project Setup



Creating React App

Introduction to React JS

Session Initial Setup

The screenshot displays the NextWave Learning Platform interface. On the left is a dark sidebar with a list of course items: 'Introduction to React JS | Cheat Sheet' (15 mins), 'Practice 1' (30 mins), 'Solving A React JS Question In Cloud IDE' (2 mins), 'Coding Practice 1' (1 hour), 'Components & Props' (40 mins), 'Components & Props | Cheat Sheet' (15 mins), and 'Practice 2' (30 mins). The main content area has a top navigation bar with a search bar (containing 'Search for \'Arrays\''), a clock icon, and links for 'Home', 'Code Playground', 'Tech Educator', and a user profile. Below the navigation bar is a video player showing a purple-themed video with a laptop illustration. The video player controls show a progress bar at 0:10 / 32:49, a language dropdown set to 'English', and icons for settings and full screen. Below the video player are links for 'Submit Feedback' and 'Ask doubt'. A tabbed interface below that has 'Notes', 'Code Playground' (selected), 'Discussions', and 'Slides'. The 'Instructions' section follows, stating: 'Go through the below steps to complete this question:'.

- Open IDE (It may take **1-2 minutes** to initialize the environment)
- Practice the concepts and skills you have learned during the session using IDE

At the bottom center, there is a blue button labeled 'OPEN IDE' enclosed in an orange rectangular box. In the bottom right corner of the main area, there is a circular icon with a robot face.

Components



React

Component

A Component is a **JS Function** that returns a **JSX Element**

JS

```
const Welcome = () => <h1 className="message">Hello, User</h1>
```

Naming Convention

Component **name** should always start with a **Capital letter**

JS

```
...  
<script type="text/babel" data-type="module">  
  import React from "https://esm.sh/react@19.1.1/?dev";  
  import ReactDOMClient from "https://esm.sh/react-dom@19.1.1/client?dev";  
  
  const Welcome = () => <h1 className="message">Hello, User</h1>  
  const root = ReactDOMClient.createRoot(document.getElementById("root"));  
  root.render(<Welcome />);  
</script>  
...
```


Creating a Component

Output

Hello, User

What if a Component name
starts with LowerCase?

Naming Convention

React treats components starting with **lowercase letters** as **HTML Elements**

JS

```
...  
<script type="text/babel" data-type="module">  
  ...  
  const welcome = () => <h1 className="message">Hello, User</h1>  
  const root = ReactDOMClient.createRoot(document.getElementById("root"));  
  root.render(<welcome/>);  
</script>  
...
```

Component Name with Lowercase

Element Panel Output

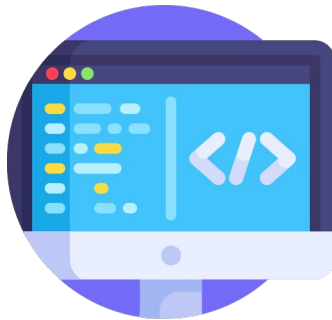
```
<!DOCTYPE html>
<html>
  ▶ <plasmo-csui> ... </plasmo-csui>
  ▶ <head> ... </head>
  ▼ <body>
    ▼ <div id="root">
      ... <welcome></welcome> == $0
    </div>
    ▶ <script type="text/babel" data-type="module"> ... </script>
  </body>
</html>
```

Component Name with Lowercase

Console Output

✖ ▶ Warning: react-dom.development.js:61
The tag <welcome> is unrecognized in this browser. If you meant to render a React component, start its name with an uppercase letter.

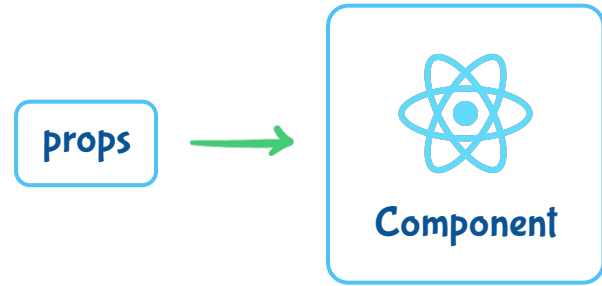
Properties (Props)



React Component

Properties (Props)

React allows us to **pass information** to a Component using **props**



React Component

Passing Props

We can pass **props** to any **Component**
as we declare **attributes** for any **HTML tag**

```
<Component propName1="propValue1" propName2="propValue2" />
```


Passing Props

JS

...

```
const Welcome = () => <h1 className="message">Hello, User</h1>
```

```
const root = ReactDOMClient.createRoot(document.getElementById("root"));
```

```
root.render(<Welcome name="Rahul" greeting="Hello" />);
```

...

Accessing Props

Components accept **props** as **parameters** and can be accessed directly

JS

```
const Component = (props) => {  
  // We can access props here  
}
```

Accessing Props

JS

```
...  
const Welcome =(props) => {  
  console.log(props);  
  return <h1 className="message">Hello, User</h1>;  
};  
const root =  
ReactDOMClient.createRoot(document.getElementById("root"));  
root.render(<Welcome name="Rahul" greeting="Hello" />);  
...
```

```
{name: "Rahul", greeting: "Hello"}  
  i  
  greeting: "Hello"  
  name: "Rahul"  
  ► __proto__: Object
```

Accessing Props

JS

```
...  
const Welcome =( props ) => {  
  const { name, greeting } = props;  
  return <h1 className="message">{greeting}, {name}</h1>;  
};  
const root =  
ReactDOMClient.createRoot(document.getElementById("root"));  
root.render(<Welcome name="Rahul" greeting="Hello" />);  
...
```

Accessing Props

Output

Hello, Rahul

React Component

Reusable

A Component is a piece of **reusable code** that can be used in various parts of an application



Reusable

JS

```
...  
  
const Welcome =( props ) => {  
  const { name, greeting } = props;  
  return <h1 className="message">{greeting}, {name}</h1>;  
};  
  
const root = ReactDOMClient.createRoot(document.getElementById("root"));  
root.render(  
  <div>  
    <Welcome name="Rahul" greeting="Hello" />  
    <Welcome name="Ram" greeting="Hi" />  
  </div>  
>);  
...
```

React Component

Output

Hello, Rahul

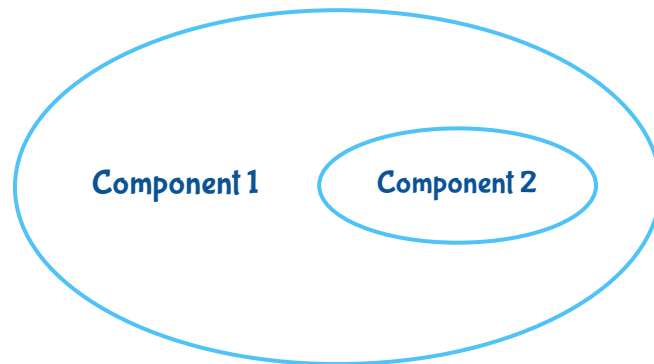
Hi, Ram

React Component

Composable

Components are composable

We can include a **component**
inside another **component**



Composable

JS

```
...  
const Welcome = (props) => {  
  const { name, greeting } = props;  
  return (<h1 className="message">{greeting}, {name}</h1>);  
}  
  
const Greetings = () => (  
  <div>  
    <Welcome name="Rahul" greeting="Hello" />  
    <Welcome name="Ram" greeting="Hi" />  
  </div>  
)  
  
const root = ReactDOMClient.createRoot(document...("root"));  
root.render(<Greetings/>);  
...
```

React Component

Output

Hello, Rahul

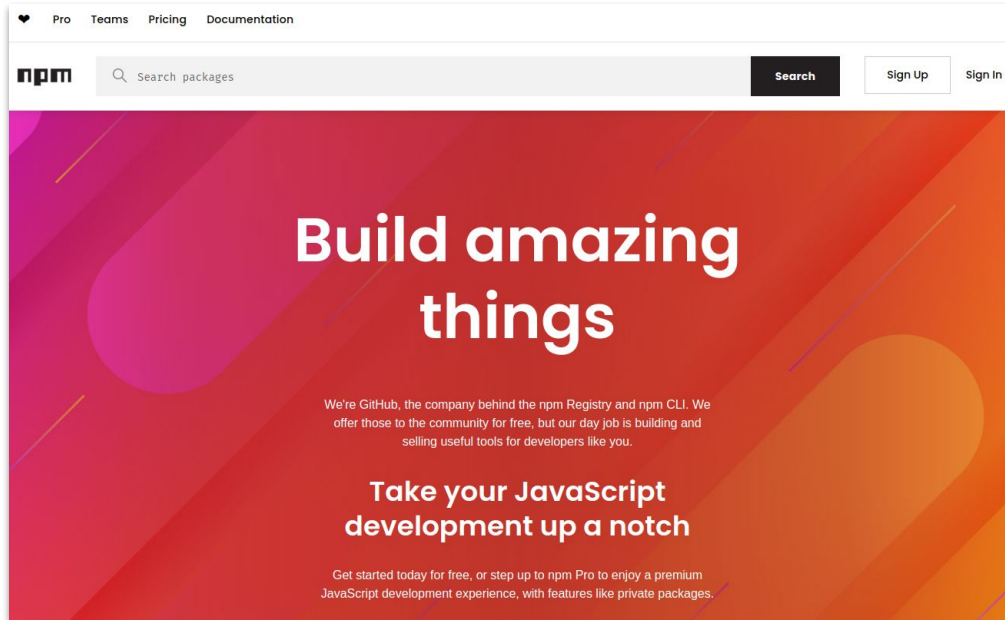
Hi, Ram

Real World React Apps

- ✓ Creating a **real-world app** involves a lot of **setup** because a large number of **components** need to be **organised**
- ✓ **Evan You**, the creator of Vue.js created a **third-party package** “**Vite**” to generate a ready-made react application setup

Third-Party Packages

NPM (Node Package Manager)



<https://www.npmjs.com/>

NPM is a package manager with millions of packages, serving as a repository and tool for managing JavaScript project dependencies

Third-Party Packages

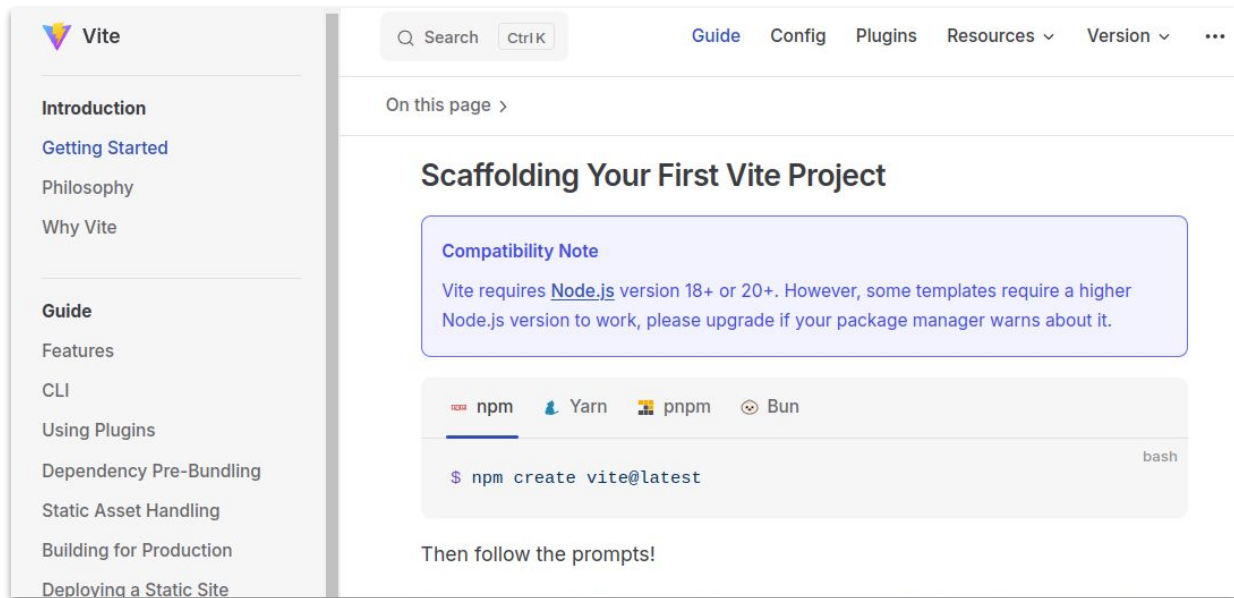
Node Package Manager

NPM Company provides a **Command line tool** which will allow you to **publish**, **discover**, **install**, and **develop** node programs



vite

Creating a React app using Vite



<https://vite.dev/guide/>

Run this command in CCBP-IDE **terminal**
to **create a React application**

Terminal

```
nxtwave@RJSIVIMLDB:~/.../$ npm create vite@8.0.2
```


Creating a React app using Vite

Terminal

```
> npx
> create-vite

◇ Project name:
  myapp

◇ Select a framework:
  React

◇ Select a variant:
  JavaScript

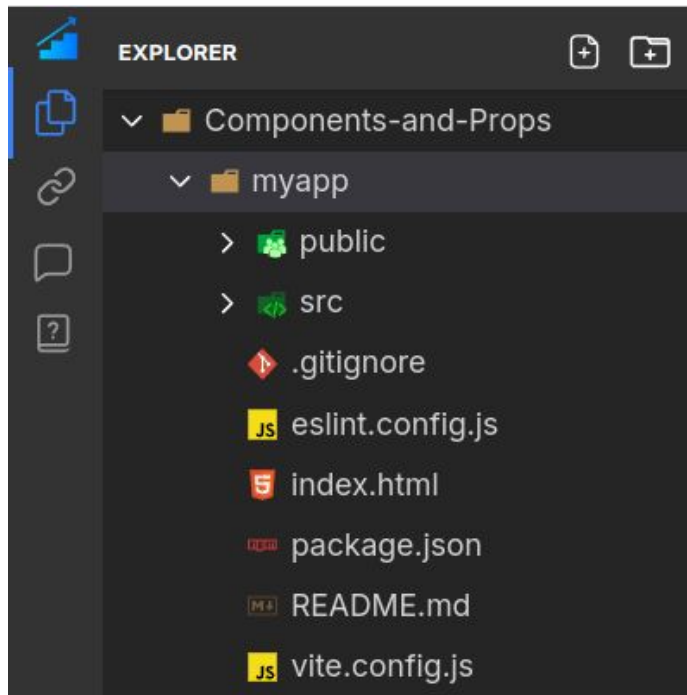
◇ Use rolldown-vite (Experimental)?:
  No

◆ Install with npm and start now?
  ● Yes / ○ No
```

It installs all the required dependencies and runs automatically

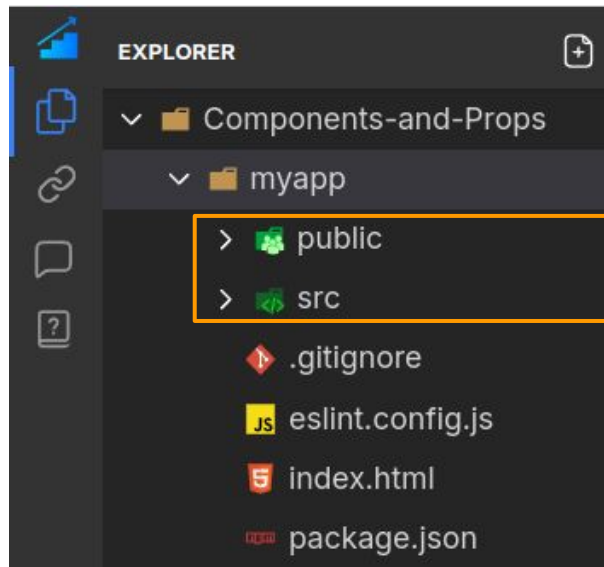
React Application

Folder Structure



Folder Structure

- ✓ **public/ folder:** Where we'll keep assets like images, icons, fonts, etc.
- ✓ **src/ folder:** Where we'll do the majority of our work. All of our React components will be placed here



IDE Specific Config

vite.config.js

```
import { defineConfig } from "vite";  
import react from "@vitejs/plugin-react";  
import eslint from "vite-plugin-eslint";
```

```
// https://vite.dev/config/  
export default defineConfig({
```

```
  server: {  
    port: 3000,  
    host: "0.0.0.0",  
    allowedHosts: true,  
  },
```

```
  plugins: [react(), eslint()],  
});
```

Here, it tells Vite to start the dev server on port 3000

Make sure to add this to check the output

IDE Specific Config

vite.config.js

```
import { defineConfig } from "vite";  
import react from "@vitejs/plugin-react";  
import eslint from "vite-plugin-eslint";
```

```
// https://vite.dev/config/  
export default defineConfig({
```

```
  server: {  
    port: 3000,  
    host: "0.0.0.0",  
    allowedHosts: true,  
  },
```

```
  plugins: [react(), eslint()],  
});
```

It's specific to only our IDE, you might not need this when running locally

myapp

Installing Dependencies

Install the dependencies from **myapp** directory

Terminal

```
nxtwave@RJSIVIMLDB:~/components-and-props/myapp$ npm install  
vite-plugin-eslint --save-dev
```

myapp

Starting an Application

Now, run `npm run dev` command to start the application

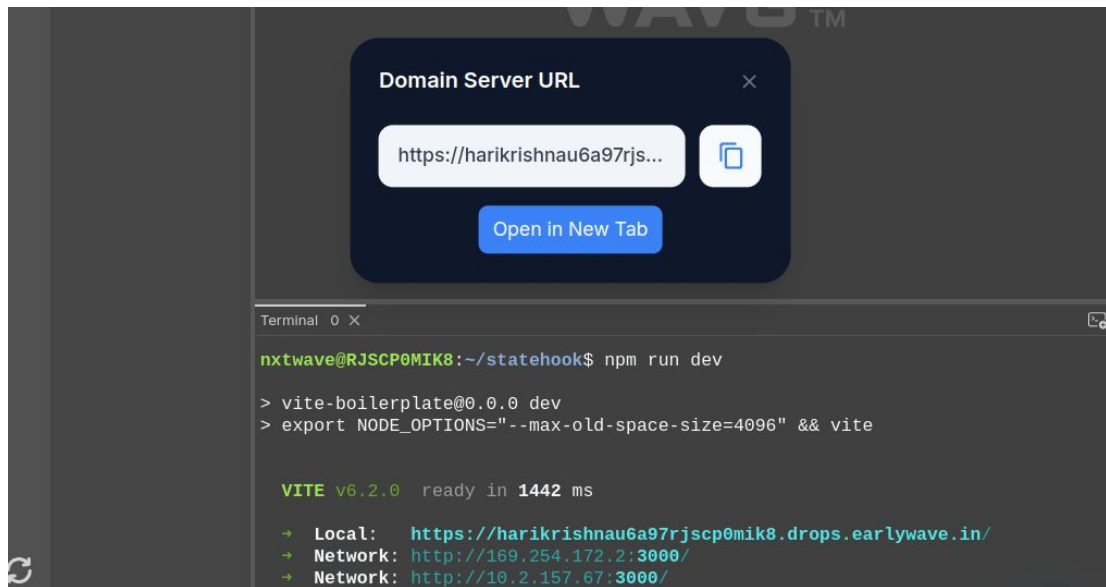
Terminal

```
nxtwave@RJSIVIMLDB:~/components-and-props/myapp$ npm run dev
```

myapp

Checking the Output

Open the URL in the new tab to check the output



The screenshot shows a terminal window with the following output:

```
Terminal 0 X
nxtwave@RJSCP0MIK8:~/statehook$ npm run dev
> vite-boilerplate@0.0.0 dev
> export NODE_OPTIONS="--max-old-space-size=4096" && vite

VITE v6.2.0 ready in 1442 ms
→ Local: https://harikrishnau6a97rjscp0mik8.drops.earlywave.in/
→ Network: http://169.254.172.2:3000/
→ Network: http://10.2.157.67:3000/
```

Overlaid on the terminal is a dark blue modal dialog titled "Domain Server URL" with a close button (X). It contains a text input field with the URL "https://harikrishnau6a97rjs...", a copy icon, and a blue button labeled "Open in New Tab".

React Application

Output



Vite + React

count is 0

Edit `src/App.jsx` and save to test HMR

Click on the Vite and React logos to learn more

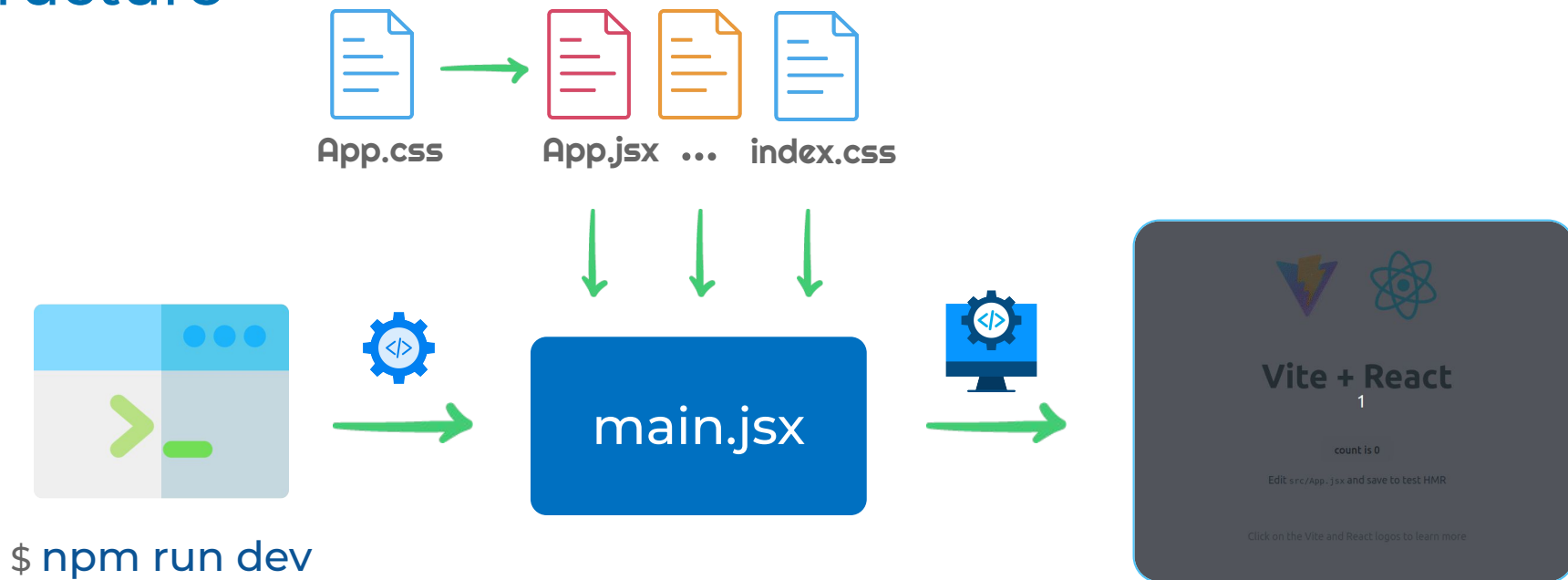
React Application

src/ folder

The `main.jsx` is a starting point to the Application

```
main.jsx ×  
1  import {StrictMode} from 'react'  
2  import {createRoot} from 'react-dom/client'  
3  import './index.css'  
4  
5  import App from './App.jsx'  
6  
7  createRoot(document.getElementById('root')).render(  
8    <StrictMode>  
9      <App />  
10    </StrictMode>,  
11  )  
12
```

React Application Structure



Folder Structure

src/ folder

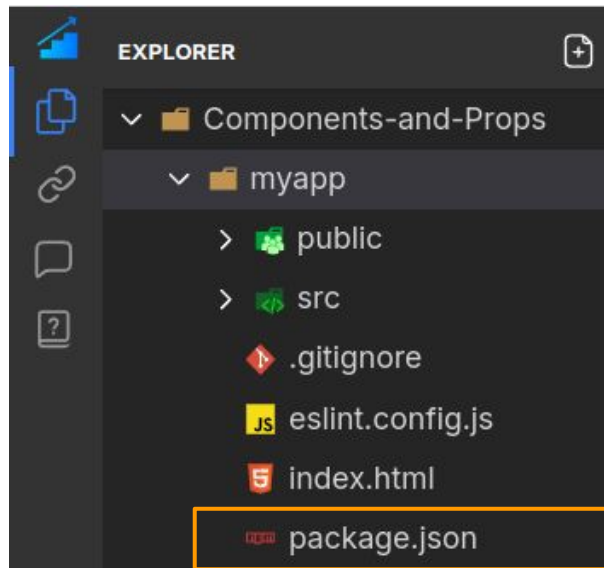
The **main.jsx** is a starting point to the Application

```
main.jsx x
1  import {StrictMode} from 'react'
2  import {createRoot} from 'react-dom/client'
3  import './index.css'
4
5  import App from './App.jsx'
6
7  createRoot(document.getElementById('root')).render(
8    <StrictMode>
9      <App />
10    </StrictMode>,
11  )
```

```
App.jsx x
1  import { useState } from 'react'
2  import reactLogo from './assets/react.svg'
3  import viteLogo from '/vite.svg'
4  import './App.css'
5
6  function App() {
7    const [count, setCount] = useState(0)
8
9    return (
10      <div>
11        <a href="https://vite.dev" target="_blank">
12          <img src={viteLogo} className="logo" alt="Vite logo" />
13        </a>
14        <a href="https://react.dev" target="_blank">
15          <img src={reactLogo} className="logo react" alt="React logo" />
16        </a>
17      </div>
18      <h1>Vite + React</h1>
19      <div className="card">
20        <button onClick={() => setCount((count) => count + 1)}>
```

Folder Structure

- ✓ **package.json:** JSON file containing the configuration for the project, including dependencies and other project details



ES6 Modules

JavaScript files containing JSX syntax should be named with `.jsx` extension

Default Exports

```
export default ComponentName
```

Named Exports

```
export const ComponentName = () => {}
```

Inspect Element Output

```
▼ <body> flex
... ▼ <div id="root"> == $0
  ▼ <div>
    ▶ <a href="https://vite.dev" target="_blank"> ... </a>
    ▶ <a href="https://react.dev" target="_blank"> ... </a>
  </div>
  <h1>Vite + React</h1>
  ▶ <div class="card"> ... </div>
  <p class="read-the-docs">Click on the Vite and React
    logos to learn more</p>
</div>
<script type="module" src="/src/main.jsx"></script>
</body>
```

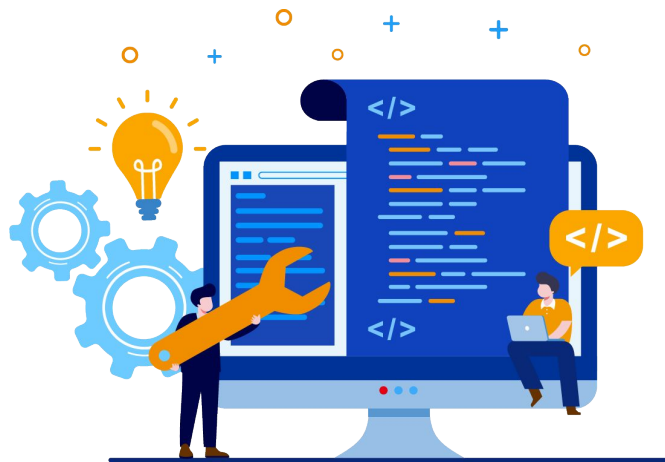
Everything gets attached
to div element with id **root**
in the **index.html**

React Application

Pre-Configured Tools

create vite comes pre-configured with

- ✓ Live editing
- ✓ esbuild
- ✓ Rollup



Pre-Configured Tools

Live Editing

Live editing allows **React components** to be live reloaded



Pre-Configured Tools

Esbuild & Rollup

These are tools used to stitch together a group of modules into a single file (or group of files)

This process is called **Bundling**



Pre-Configured Tools

esbuild

esbuild is also a **JavaScript Compiler**,
compiles **JSX** into **Regular JavaScript**

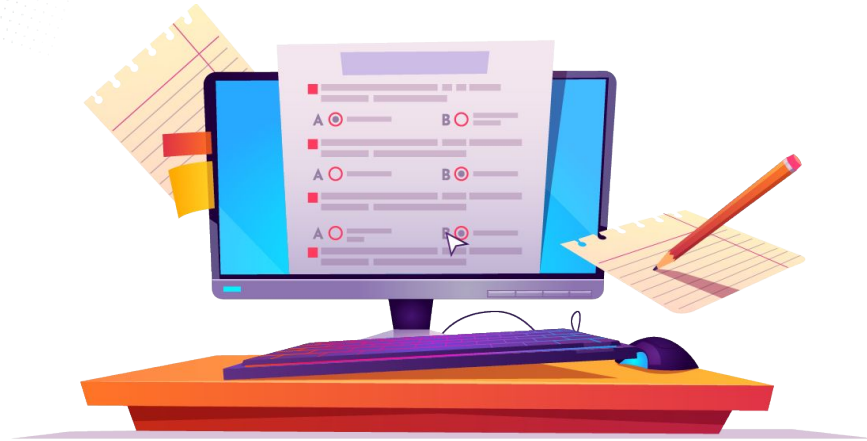


Key Takeaways

- ✓ Creating React App using Vite
 - React Project Setup
 - Starting the Application



Practice





Next Session

Lists & Keys



THANK YOU





ALL THE BEST

